Micro-app benchmark and optimization report

Branch: `perf/benchmark-and-optimize`. Machine: WSL2 laptop, Intel Core Ultra 7 268V (8 cores, no SMT), 15 GB RAM, no GPU. Toolchain: GCC 16, OpenMPI 5.0, Kokkos 5.2.2 (Serial + OpenMP), PETSc 3.26 built from source with Kokkos/kokkos-kernels, HighFive 3.3, Release build with `-O3 -march=native` for PETSc and CMake Release flags for the apps. All timings are wall-clock, best of repeated runs on an otherwise idle machine unless noted.

The two figures below show where the time went before any changes and the headline speedups.

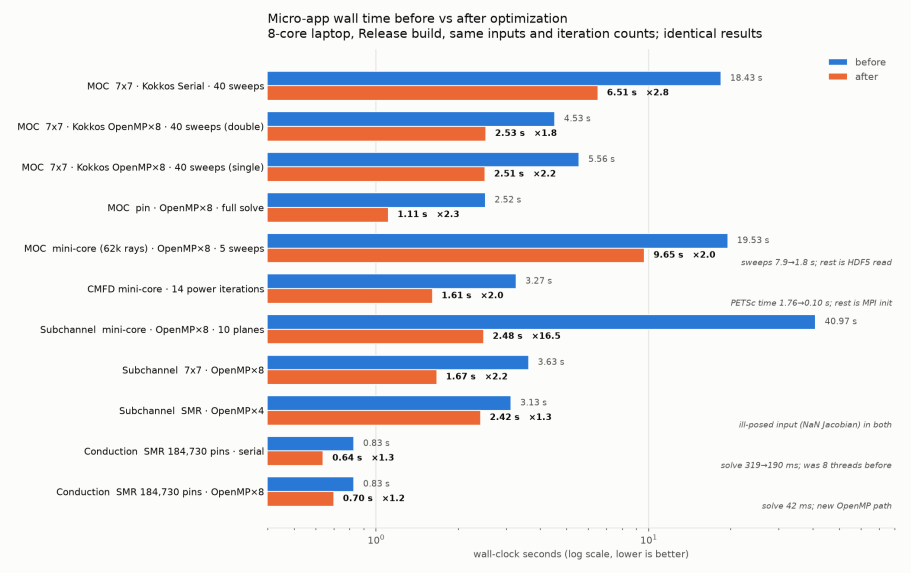
Summary

Same inputs, same iteration counts, results identical to the baseline (see Correctness). Wall-clock seconds.

App	Case	Before	After	Speedup	Dominant remaining cost
MOC	7x7, Kokkos Serial, 40 sweeps	18.43	6.51	2.8x	sweep (0.11 s/sweep) + 1.6 s HDF5 ray read
MOC	7x7, Kokkos OpenMP x8, 40 sweeps, double	4.53	2.53	1.8x	HDF5 ray read (1.6 s)
MOC	7x7, Kokkos OpenMP x8, 40 sweeps, single	5.56	2.51	2.2x	HDF5 ray read
MOC	pin, OpenMP x8, full solve to 1e-8	2.52	1.11	2.3x	
MOC	mini-core (62k rays), OpenMP x8, 5 sweeps	19.53	9.65	2.0x	HDF5 ray read (~7 s); sweeps 7.9 -> 1.8 s (4.3x)
CMFD	mini-core, 14 power iterations (best of 3)	3.27	1.61	2.0x	MPI_Init (~1.5 s); PETSc-logged time 1.76 -> 0.10 s (17x)
CMFD	pin / 7x7 / small_parallel	~1.55	~1.55	1.0x	entirely MPI_Init; solver time 0.06 -> 0.02 s on 7x7

App	Case	Before	After	Speedup	Dominant remaining cost
Subchannel	mini-core (800 ch, 1540 surf), OpenMP x8, 10 planes	40.97	2.48	16.5x	MPI_Init + geometry setup
Subchannel	7x7, OpenMP x8	3.63	1.67	2.2x	startup (no Jacobian needed in this case)
Subchannel	SMR (832 ch), OpenMP x4, 29 planes	3.13	2.42	1.3x	input is ill-posed in both versions (see below)
Conduction	SMR, 184,730 pins, serial	0.83	0.64	1.3x	HDF5 load + libmpi load; solve 319 -> 190 ms, and the "serial" baseline was really 8 threads (true 1-thread solve: ~560 -> 190 ms, 2.9x)
Conduction	SMR, OpenMP x8 (new path)	-	0.70		solve 42 ms

Test-suite time (ctest, all labels) is also much lower: CMFD 498 s -> 37 s, the MOC OpenMP single-precision 7x7 test 214 s -> 7 s (under load), subchannel mini-core/watts-bar tests from tens of seconds to a few.



Build fixes required before anything would compile

The code did not build against current Kokkos 5 / HighFive 3:

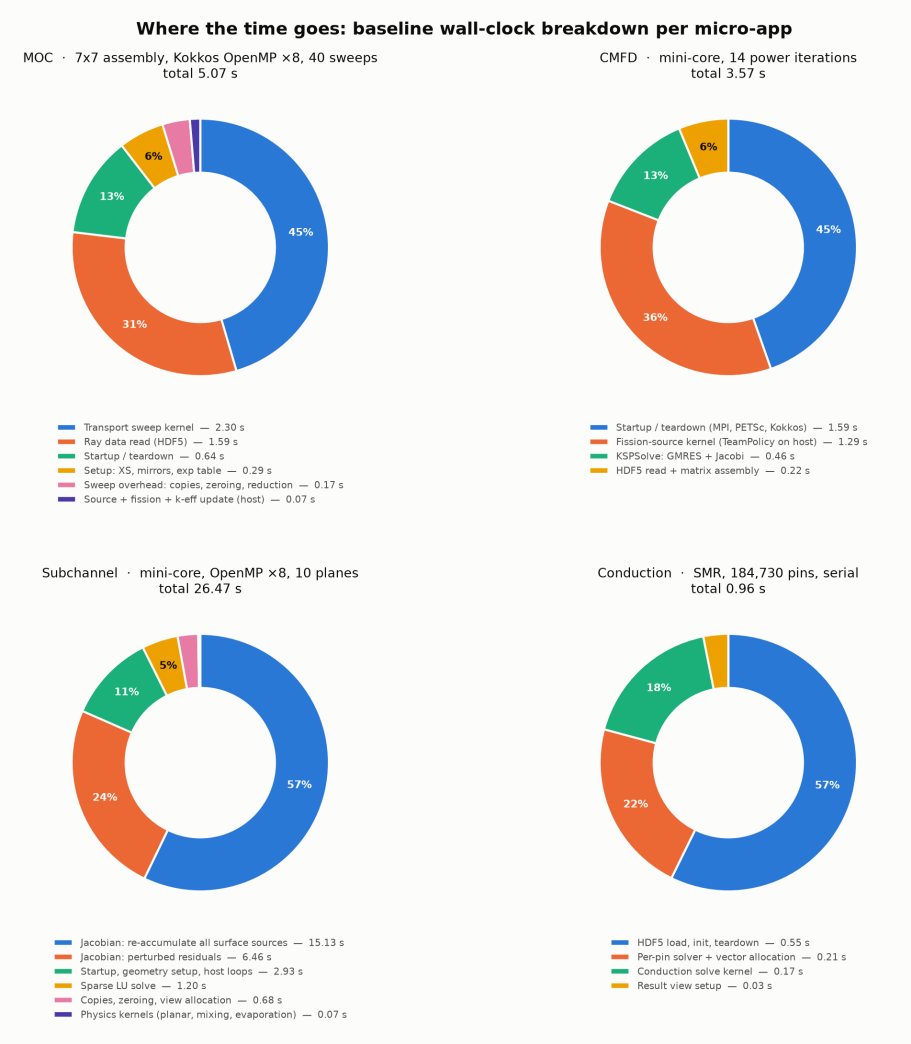
- `ExecutionSpace::concurrency()` is an instance method in Kokkos 5 (`moc/src/kokkos_moc.cpp`, `subchannel/src/solver.cpp`).
- `View::HostMirror` was removed (`utils/src/hdf5_kokkos.hpp`, CMFD tests).
- `HighFive::DataSet::read(T*)` was replaced by `read_raw` (`utils/src/hdf5_kokkos.hpp`, CMFD tests).
- `cmake/FindPETSc.cmake` searched `CMAKE_PREFIX_PATH` for `petsc.h` and could pick up a different PETSc than the one in `PETSC_DIR`.
- The top-level `CMakeLists.txt` had no default build type, so following the README produced `-O0` binaries. It now defaults to Release.

Baseline profile (what dominated before)

Measured with a small Kokkos-Tools profiler (per kernel/region/deep-copy timing) and PETSc `-log_view`.

- **MOC** (7x7, Kokkos OpenMP x8, 40 sweeps, 5.07 s): sweep kernel 2.30 s, HDF5 ray read 1.59 s, process startup 0.64 s, setup 0.29 s. The hand-written serial sweeper (0.137 s/sweep) was 3.5x faster than the Kokkos sweeper on one core (0.48 s/sweep) and faster than the Kokkos sweeper on 8 threads (0.17 s/sweep).
- **CMFD** (mini-core, 14 power iterations, 3.57 s): MPI/PETSc startup 1.59 s, fission-source kernel 1.29 s (a `TeamPolicy(nCells, nGroups)` with nested team reductions, pathological on the OpenMP backend: 86 ms per call for 7 x nCells values), KSPSolve 0.46 s (1599 GMRES iterations over 14 solves, zero initial guess every time), HDF5 read + assembly 0.22 s.
- **Subchannel** (mini-core, OpenMP x8, 10 planes, 26.5 s): finite-difference Jacobian 21.6 s. For each of the 1540 surfaces, serially: copy `gk` to host and back twice, zero and re-accumulate all surface sources, then evaluate 7 residuals. That is O(nsurf^2) work plus ~6 kernel launches and 4 host/device copies per column, into a dense nsurf x nsurf Jacobian that was converted to CSR by scanning it on the host.
- **Conduction** (SMR, 184,730 pins, 0.96 s): HDF5 load/init/teardown 0.55 s, per-pin heap allocation 0.21 s, actual solve kernel 0.17 s. The "serial" test was actually running on 8 OpenMP threads (integer-range `parallel_for` uses the default execution space) and had an out-of-bounds write (see below).

A fixed cost outside the apps: `MPI_Init` through conda OpenMPI 5 takes 1.5-1.9 s per process on this machine because UCX transports are probed; `OMPI_MCA_pml=ob1 OMPI_MCA_btl=self` brings it to 0.4 s. This dominates the small CMFD and subchannel cases and is an environment setting, not a code change.



Changes by app

MOC (`moc/src/kokkos_moc.{hpp,cpp}`, `moc/src/eigen_solver.{hpp,cpp}`)

1. **Host-specialized sweep** (`_sweep_host`): on Serial/OpenMP each work item is a (ray, polar) pair and the energy-group loop is innermost, so source, cross-section and flux-tally accesses for one FSR are contiguous runs instead of strided gathers. Exponential arguments are computed once per segment and reused by the backward pass. Per-thread tallies (kept in double) replace atomics; a parallel kernel reduces them. Dynamic scheduling balances the very different ray lengths. The GPU kernel (`_sweep_device`) is unchanged.
2. **Angular-flux buffer swap**: the sweep used to copy old_angflux host->device twice, then angflux device->host and host->host every sweep. The two device buffers are now swapped.

3. **Source and fission source on the device**: `update_source` and `fission_source` were serial host loops followed by a full (nfsr x ng) host->device copy; they are now kernels, and only the nfsr-sized fission source crosses the host/device boundary.
4. **Eigen solver**: removed a per-iteration `scalar_flux()` copy into `vector<vector<double>>` and a relaxation loop whose result was never used; the fission source is rescaled to the new keff instead of being recomputed.
5. **Ray setup**: each ray's FSR dataset was read twice from HDF5; it is now read once.

CMFD (`cmfd/src/PetscMatrixAssembler.{hpp,cpp}`, `cmfd/src/PetscEigenSolver.{hpp,cpp}`)

1. **Fission-source kernels** (COO and CSR assemblers): the `TeamPolicy(nCells, nGroups)` with a nested `TeamThreadRange` reduction and team barrier per cell is now a flat `RangePolicy` over cells with serial loops over the ~7 groups. Per call on mini-core: ~74 ms -> ~10 us. The kernels capture only the three views they need instead of the whole `CMFDData`, remain plain `KOKKOS_LAMBDA`s (CUDA-portable), and the group sum is now deterministic. Under load the old kernel degraded to 12 s per run; the new one is unaffected.
2. **Inner solve starts from the previous flux** (`KSPSetInitialGuessNonzero`) and the two `VecDot` calls are one `VecMDot`. Total KSP iterations on mini-core: 1543 -> 1080; 7x7: 534 -> 271. Semantics change, flagged: with a nonzero guess and the inner relative tolerance equal to the outer keff tolerance (1e-7), the inner solve stops after ~0 iterations once the flux changes by less than 1e-7 and the power iteration stalls 1.2e-7 short of the gold keff. The inner tolerance is therefore `tol/10`. Outer convergence criterion and test tolerances are unchanged; 7x7 keff error vs gold went from 8.1e-8 to 1.7e-8.
3. **PETSc object lifetime fixes**: the CSR assembler created a new fission `Vec` on every call (leaking the previous one and the one from `initialize()`); it is now created once and refilled through `VecGetKokkosViewWrite`, so the `Vec` cached by the eigen solver stays valid and GPU host/device state is tracked correctly. The COO assembler ran `VecSetPreallocationCOO` every call; now once. A leaked `MatCreate` in the CSR matrix assembly is removed. Copying an assembler (which double-destroyed the shared `Vec`/`Mat`) is deleted and a move constructor transfers ownership; `test_coarseVsFineEigenvalue` relied on the old recreate-every-call behavior to survive the member-wise copy.
4. Solver/preconditioner sweep (not adopted, GMRES + Jacobi kept as the GPU-friendly default): BiCGStab variants were no faster; SOR halves the iterations but is not GPU-friendly; GAMG needs 80 iterations total but its setup (0.49 s) outweighs the gain at this size; an adaptive inner tolerance tied to |delta keff| cuts matvecs by ~30% on mini-core but costs two extra outer iterations on 7x7.

CMFD tests: 10/10 pass; the suite went from 498 s to 37 s (PetscEigenSolverHeavy 117 s -> 5 s, coarseVsFineEigenvalue 342 s -> 3.3 s).

Subchannel (`subchannel/src/th.{hpp,cpp}`, `geometry.{hpp,cpp}`, `petsc_linear_solver.{hpp,cpp}`)

1. **Single-kernel sparse Jacobian**: perturbing `gk` on surface ns1 only changes the sources of its two channels, so column ns1 is formed from those two channels' source deltas (`surface_source_terms`, shared with the normal accumulation) and the residuals of the <= 7 neighbor surfaces. All columns run in one `parallel_for`; nothing is copied to the host inside the Newton loop. Work per Newton step goes from O(nsurf^2) to O(7 nsurf).
2. **CSR Jacobian layout from the geometry**: `Geometry::build_surface_connectivity` now also builds the CSR row pointer, column indices and a (ns1, n) -> position map, so the kernel writes straight into CSR values. The dense nsurf x nsurf Jacobian (and its host-side O(nsurf^2) scan to CSR) is gone; memory for the Jacobian drops from 8 nsurf^2 bytes to ~56 nsurf bytes.
3. **Linear solve from CSR** (`PetscLinearSolver::solve_csr`): on host execution spaces a plain SeqAIJ matrix is used so PETSc's native sparse LU does the triangular solves (the Kokkos-Kernels team-based sptrsv was ~1 s of the baseline); device spaces keep `MatCreateSeqAIJKokkosWithKokkosViews`.
4. **Device-side `gk` handling**: the initial guess copy from the previous plane, the max-residual check and the Newton update are kernels/reductions instead of full host copies of `gk`.
5. **Geometry accessors**: `dz()`, `global_chan_index()`, `nassemblies()` etc. created and deep-copied a host mirror of a whole table on every call and were called inside setup loops over all channels; they now read cached host mirrors. The file-based constructor also never copied `_ij_global`, `surfaces` and `_ns_global` from their host mirrors to the device views (harmless on CPU, wrong on GPU); it does now.

Results match the baseline: identical final exit void / pressure-drop tables for the 3x3 and 7x7 cases and an identical final-plane state summary for the mini-core case; Newton iteration counts per plane differ by at most one on two planes.

Conduction (`conduction/src/...`)

1. **Allocation-free hot loop**: the per-pin solve works on stack arrays (two `CylinderNode` copies, interface temperatures, workspace) through a templated core in `cylindrical_solver_detail`; the public `CylindricalSolverSerial` class is a thin wrapper over the same code so the unit tests exercise identical arithmetic. Radius logarithms and volumes are cached per inner iteration.

2. **Correct execution space**: `Kokkos::parallel_for(name, N, lambda)` runs on the default execution space, so `Conduction<Kokkos::Serial>` was actually running on 8 OpenMP threads. It now uses `RangePolicy<execution_space>`. OpenMP is verified bitwise identical to Serial (new `test_smr_omp`).
3. **Out-of-bounds write fixed**: the interface-temperature view had `num_nodes + 1 = 3` columns but a fuel/gap/clad pin has 4 interfaces, so every pin wrote its last value into the next pin's row (a race under OpenMP, and 8 bytes past the allocation for the last pin). `interface_temperatures[i]` now has 4 entries.
4. HDF5 file opened once per load instead of once per dataset; exceptions inside the parallel region are captured and rethrown instead of terminating.

Correctness

- Full `ctest` run on the optimized build (all labels, 49 tests including the HEAVY MOC mini-core serial cases): **100% passed** (1500 s total, dominated by the two hand-written-serial-sweeper mini-core tests).
- MOC: keff per iteration identical to 8 printed decimals vs the baseline for 7x7 and mini-core in double; the OpenMP single-precision 7x7 gold test (tolerance 1e-7) passes with the double tallies.
- CMFD: all gold-keff tests pass; 7x7 error vs gold improved from 8.1e-8 to 1.7e-8.
- Subchannel: exit void / pressure-drop tables identical for 3x3 and 7x7; final-plane state summary identical for mini-core; Newton iteration counts per plane identical except two planes that differ by one.
- Conduction: average temperatures and the first three interface temperatures bitwise identical to a race-free serial run of the baseline; the fourth interface temperature (clad outer surface) is new output. OpenMP is bitwise identical to Serial.

Things found but not changed

- **MPI startup**: 1.5-1.9 s of every CMFD/subchannel process is `MPI_Init` probing UCX transports (`OMPI_MCA_pml=ob1 OMPI_MCA_btl=self` -> 0.4 s). Even an executable that never calls MPI pays ~0.4 s to load libmpi/UCX because conda's parallel HDF5 and PETSc's Kokkos link it. Environment/packaging, not code.
- **MOC ray read**: each ray is an HDF5 group with four datasets; 62k rays = 310k object opens = ~7 s. Only a different file layout (one dataset per angle with offsets) would fix this.
- **MOC device sweep**: the CUDA/SYCL kernel is unchanged. Note that `_configure_team_policy` launches `TeamPolicy(n, 1, 1)`, i.e. teams of one thread; on CUDA that is one thread per block. A `RangePolicy` over (ray, polar, group) with the same body would expose the same parallelism with full warps and is worth trying on a GPU.

- **MOC single precision**: the OpenMP single-precision 7x7 gold (1e-7 tolerance) is beneath float summation noise; the per-thread tallies were made double so results do not depend on thread scheduling.
- **Subchannel SMR input**: the Jacobian has rows that are all zero/NaN (channels with zero flow area), so the baseline hit PETSc zero-pivot errors (324 error lines, 15 failed solves) and produced NaN pressures; the new code reports KSP_DIVERGED_NANORINF cleanly but the result is equally meaningless. Needs a physics fix.
- **Subchannel quality in the Jacobian**: `solve_quality` reads `W_l(ij,k)`/`W_v(ij,k)` from the state views rather than the perturbed local flow rates, so quality does not respond to the perturbation. Preserved for identical results; probably a bug.
- **CMFD CSR matrix**: rows have unsorted columns and zero-valued padding entries in column 0 for boundary cells. Jacobi/MatMult tolerate it, but `-pc_type ilu` aborts and `-pc_type pbjacobi` gives NaN; ILU on a cleaned matrix cut matvecs by 3x. Also the KSP convergence reason is never checked, and the keff convergence test compares against the value two iterations back.
- **Conduction**: `ConductionResults::interface_temperatures` is a `vector<vector<double>>` (184k small allocations outside the hot loop); `cylindrical_solver_base.*` is dead code.
- **Geometry (subchannel)**: `nsurfaces()` still copies the core map on each call (used only at setup).

Reproducing

`perf/tools/bench.sh` runs the benchmark set; `perf/tools/kptimer.cpp` is the profiler used for the breakdowns; the two chart scripts regenerate the figures.